WEEK 3 · LESSON 13 OF 24 · HANDOUT

Images as Numbers and Convolutions

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=py5byOOHZM8 — open the lesson online to watch inside the course.

🎓 **Go deeper:** MIT 6.S191 (2024): Convolutional Neural Networks — https://www.youtube.com/watch?v=2xqkSUhmmXU

🎓 **Go deeper:** Harvard CS50 AI, Lecture 5: Neural Networks — https://www.youtube.com/watch?v=mFZazxxCKbw

Your goal in this lesson: Describe how images are stored as tensors, apply a convolution filter by hand and with PyTorch, and explain feature maps, padding, stride and pooling.

An image is a grid of numbers

A grey image is a 2-D grid of **pixels**; each pixel is a brightness number (0 = black, 1 or 255 = white). A colour photo has **three channels** — red, green and blue — so it is a 3-D block. PyTorch stores a batch of images as a 4-D tensor:

> *(batch size, channels, height, width)*

A phone photo of a cassava leaf at 224 × 224 pixels is 3 × 224 × 224 = **150,528 numbers**.

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

digits = load_digits()
img = torch.tensor(digits.images[0] / 16.0, dtype=torch.float32)
print(img.shape)
print(img.round(decimals=1))
plt.imshow(img, cmap="gray_r")
plt.title(f"label {digits.target[0]}")
plt.show()
```

Why not use a plain network?

In Week 1 we **flattened** the digit into 64 numbers. That throws away the layout — the network no longer knows which pixels are neighbours — and for big photos it needs a weight for every pixel in every neuron: a 150,528-pixel photo into just 1,000 neurons is 150 million weights. Worse, a leaf spot in the top corner and the same spot in the middle would look like completely different inputs.

A **convolutional neural network (CNN)** fixes all three problems with one idea.

The convolution

A **filter** (or **kernel**) is a small grid of weights — for example 3 × 3. It **slides** across the image; at every position it multiplies the 9 pixels under it by its 9 weights and adds them up. The result is a new image called a **feature map**, which lights up wherever the pattern in the filter appears.

Let's design a vertical-edge filter by hand:

```
edge = torch.tensor([[-1.0, 0.0, 1.0],
                     [-1.0, 0.0, 1.0],
                     [-1.0, 0.0, 1.0]])

# a 6 x 6 image: dark on the left, bright on the right
image = torch.zeros(6, 6)
image[:, 3:] = 1.0

out = F.conv2d(image.view(1, 1, 6, 6), edge.view(1, 1, 3, 3))
print(out.squeeze())
```

The output is **3** in the two columns where dark meets bright and **0** everywhere else: the filter found the edge. Notice the output is 4 × 4, smaller than the input — a 3 × 3 filter cannot be centred on the border pixels.

Filters on a real digit

```
digit = img.view(1, 1, 8, 8)
vertical = F.conv2d(digit, edge.view(1, 1, 3, 3), padding=1)
horizontal = F.conv2d(digit, edge.T.reshape(1, 1, 3, 3), padding=1)

fig, axes = plt.subplots(1, 3, figsize=(8, 3))
for ax, im, title in zip(axes, [digit, vertical, horizontal], ["digit", "vertical edges",
"horizontal edges"]):
    ax.imshow(im[0, 0].detach(), cmap="gray")
    ax.set_title(title)
    ax.axis("off")
plt.show()
```

In a CNN **nobody designs the filters** — they are weights, learned by backpropagation like any other. The first layers usually learn edge and colour detectors, the next layers corners and textures, deeper layers parts of objects (a leaf vein, an eye, a wheel).

Three big advantages

1. **Few weights:** one 3 × 3 filter has 9 weights, whatever the image size.
2. **Weight sharing:** the same filter is used at every position, so a pattern is detected **wherever** it appears.
3. **Local patterns:** each output only looks at a small neighbourhood, which matches how images work.

Padding, stride and channels

- **Padding** adds a border of zeros so the output keeps the input size (`padding=1` for a 3 × 3 filter).
- **Stride** is the step size of the slide; stride 2 halves the width and height.
- A conv layer has **many filters**: `nn.Conv2d(1, 16, 3)` learns 16 filters and outputs 16 feature maps (channels).

```
conv = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)
print(conv(digit).shape)                      # 16 feature maps of 8 x 8
print("weights:", conv.weight.shape, "-> parameters:", sum(p.numel() for p in
conv.parameters()))
strided = nn.Conv2d(1, 16, 3, padding=1, stride=2)
print(strided(digit).shape)                   # stride 2 halves the size
```

16 filters × 9 weights + 16 biases = **160 parameters** for the whole layer.

Pooling: shrink and keep the strongest signal

Max pooling takes each 2 × 2 block and keeps only its largest value. It halves the width and height, makes the network faster, and makes it less sensitive to small shifts.

```
x = torch.tensor([[1.0, 3.0, 2.0, 0.0],
                  [4.0, 2.0, 1.0, 1.0],
                  [0.0, 1.0, 5.0, 6.0],
                  [2.0, 2.0, 7.0, 1.0]]).view(1, 1, 4, 4)
print(nn.MaxPool2d(2)(x).squeeze())
```

The four 2 × 2 blocks become **4, 2, 2 and 7**.

Key terms

- **Pixel / channel** — one brightness value / one colour layer (RGB = 3 channels).
- **Filter (kernel)** — a small grid of learned weights that slides over the image.
- **Convolution** — sliding the filter and computing weighted sums.
- **Feature map** — the output image showing where a pattern appears.
- **Padding / stride** — border of zeros / step size of the slide.
- **Max pooling** — keeping the largest value in each block.

Remember this: A convolution slides small learned filters over an image to detect patterns anywhere; stacking convolutions and pooling builds from edges up to whole objects.

Name: _____ Date: _____

Activity

Task 1

Make a horizontal-edge test image (dark top, bright bottom) and apply both filters.

Task 2

Design a 3 × 3 "blur" filter (all weights 1/9) and apply it to the digit.

Task 3

What is the output size of a 3 × 3 convolution with no padding on a 32 × 32 image? With padding 1 and stride 2?

Task 4

Count the parameters of `nn.Conv2d(3, 64, kernel_size=3)` (a colour image in, 64 filters out).

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. How does PyTorch store a batch of images?

   A. As a list of strings
   B. (width, batch)
   C. (batch, channels, height, width)
   D. (height, width) only

2. What does a convolution filter do?

   A. Deletes pixels
   B. Sorts the pixels
   C. Changes the labels
   D. Slides over the image computing weighted sums to detect a pattern

3. Who designs the filters in a CNN?

   A. They are random forever
   B. The camera
   C. The programmer, by hand
   D. They are learned by backpropagation

4. Why does weight sharing help?

   A. It makes images bigger
   B. It sorts the classes
   C. It removes the need for data
   D. The same filter detects a pattern wherever it appears, with few weights

5. What does 2 × 2 max pooling do to a 16 × 16 feature map?

   A. Leaves it unchanged
   B. Removes all channels
   C. Makes it 8 × 8, keeping the largest value of each block
   D. Makes it 32 × 32

6. How many parameters does nn.Conv2d(1, 16, 3) have?

   A. 160
   B. 9
   C. 144
   D. 16

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.